

GPSDO v1.06 — The Complete Manual (from zero to locked)

English | Polski | Español

Firmware: GPSDO v1.06-rtos by jmnlabs (after André Balsa's original GPSDO, Lars Walenius' PI loop, Alan Cashin's multi-level accumulator) **Board:** WeAct BlackPill STM32F411CE · **OCXO:** 10 MHz (e.g. Vectron C4550) **GPS:** u-blox receiver on Serial1 (LEA-M8T / NEO-M8T / ZED-F9T or NEO-6M/7M)

Author: Jarosław Marek Niewiński (jmnlabs) — **Assistant:** GLM-5.3 Max (Z.ai), including the Polish and Spanish translations of this manual This manual assumes **nothing**. If you have never built firmware, never flashed a microcontroller, and never used a serial terminal, start at Part 1 and do exactly what it says, in order. Every step tells you what to type and what you should see. Nothing important is left as an exercise.

The 60-second summary: install Arduino IDE with the STM32 core (not 3.0.0), open the sketch, pick your display in `gpsdo_config.h`, compile with *Newlib Nano + Float printf*, flash with ST-Link or DFU. **Never do a full-chip erase** — your settings and calibration live in flash sector 7 and only a full-chip erase can destroy them. Then, over serial at 115200 baud: run `CT`, wait, run `LC`, wait, run `LA 12`, run `SAW 1`, run `ES`. Watch the phase stay near zero. Done.

Table of contents

- Part 1 — Building the firmware
- Part 2 — Flashing, and the sector 7 rule
- Part 3 — First start: calibration, algorithm, save
- Part 4 — The thirteen algorithms (0–12) and their parameters
- Part 5 — Displays: what every field means
- Part 6 — Serial telemetry (the 6-line report)
- Part 7 — Command reference (every command)
- Part 8 — The PC tuner
- Part 9 — Settings, the flash ring, and wear
- Part 10 — Troubleshooting
- Appendix A — How a GPSDO works, in plain words
- Appendix B — PID for the reluctant

Part 1 — Building the firmware

1.1 What you must install

- 1. **Arduino IDE** (1.8.x or 2.x — both work).
- 2. **The STM32duino core, version 2.2.0 through 2.12.0.** Boards Manager → search "stm32" → **STM32 MCU based boards by STMicroelectronics**. Install 2.12.0 (the last supported).

WARNING — core 3.0.0 does not work. It was released in July 2026 and breaks this project three ways: `1toa()` no longer exists (compile error), `HardwareSerial Serial12(PA3, PA2)` no longer compiles, and `TFT_eSPI` stops driving the panel (permanent white screen). Stay on 2.12.0 or older until this project announces otherwise.

1. **Libraries** (Library Manager, install by exact name):

Library	Why
STM32duino FreeRTOS	the operating system
TinyGPS++	GPS sentence parsing
U8g2	OLED displays (only if you enable one)
Adafruit AHTX0	AHT10/AHT20 temperature/humidity sensor
Adafruit BMP280	BMP280 pressure/temperature sensor
Adafruit INA219	supply voltage/current monitor
hd44780	20x4 character LCD (only if you enable it)
TFT_eSPI	SPI TFT display

1. Nothing else. There is no EEPROM library — settings live in on-chip flash (Part 2).

1.2 Open the sketch

Unzip the project. Open `GPSDO_FreRTOS/GPSDO_FreRTOS.ino` in Arduino IDE. The folder must keep its name — Arduino requires the folder name to match the `.ino` file.

1.3 Pick your hardware in `gpsdo_config.h`

This one file decides what gets compiled. Lines that are active start with `#define`; lines that are off start with `//`. Uncomment exactly what your board has, comment out what it does not. The shipped configuration is a fully populated board (big TFT + all sensors); change it to match reality.

Display — pick exactly one main display:

Flag	Hardware
<code>GPSDO_TFT_ILI9488</code>	320x480 SPI TFT (shipped default, used in landscape 480x320)
<code>GPSDO_TFT_ST7789</code>	240x320 SPI TFT
<code>GPSDO_TFT_ILI9341</code>	240x320 SPI TFT
<code>GPSDO_OLED_SH1106</code> / <code>_SSD1306</code> / <code>_SSD1309</code>	128x64 I2C OLED (one only)
<code>GPSDO_LCD_20x4</code>	HD44780 20x4 via PCF8574 I2C backpack
<code>GPSDO_TM1637</code> / <code>GPSDO_TM1637_6</code>	4-digit / 6-digit LED clock (mutually exclusive, and not together with the LCD)
<code>GPSDO_HT16K33</code>	4-digit I2C LED clock (shipped default ON; independent of the main display)

Sensors and extras (all shipped ON — comment out what you don't have):

Flag	Hardware
GPSDO_AHT10	AHT10/AHT20 on I2C
GPSDO_BMP280_I2C	BMP280 on I2C
GPSDO_INA219	INA219 on I2C
GPSDO_VCC / GPSDO_VDD	measure the 5 V rail / 3.3 V rail
GPSDO_UBX_CONFIG	put NEO-6M/7M into the right binary mode at boot
GPSDO_FAKE_UBLOX	Chinese u-blox clone : baud probe only, no UBX config at all (see Part 3.2)
GPSDO_GPS_TIMING	timing-receiver support (LEA-6T/M8T/NEO-M8T/ZED-F9T): survey-in, qErr
GPSDO_PICDIV	picDIV divider arming output on PB3
GPSDO_LTIC	the phase detector hardware (ramp TIC on PA1) — required for algorithms 10, 11, 12
GPSDO_LTIC_ACTIVE_RESET	active capacitor discharge variant of the detector (leave OFF for the classic RC Kaashoek detector)
GPSDO_PWM_DITHER	24-bit control voltage from a dithered 13-bit PWM (leave ON — this is the good DAC)
GPSDO_DAC_EXT	external SPI DAC — a stub, refuses to compile by design
GPSDO_GEN_2kHz_PB5	2 kHz test square wave on PB5
GPSDO_BLUETOOTH	HC-06 Bluetooth module on PA2/PA3
GPSDO_BLUETOOTH_PARALLEL	USB and Bluetooth at the same time (shipped ON)

The file refuses to compile (on purpose) if you pick two displays of the same kind or two things that claim the same pins. Read the `#error` message — it names the conflict.

Leave `GPSDO_LTIC` and `GPSDO_PWM_DITHER` on unless you truly don't have the detector hardware: without LTIC you lose algorithms 10–12, which are the whole point of v1.06.

1.4 Configure TFT_eSPI (TFT users only)

TFT_eSPI is configured **inside the library**, not in the sketch. Find `Arduino/libraries/TFT_eSPI/User_Setup.h` and make it contain the block for your panel:

240x320 (ST7789 or ILI9341):

```
#define ST7789_DRIVER           // or ILI9341_DRIVER
#define TFT_WIDTH  240
#define TFT_HEIGHT 320
#define TFT_MISO PA6           // required on STM32 even if the display has no MISO pin
#define TFT_MOSI PA7
#define TFT_SCLK PA5
#define TFT_CS  PB13
#define TFT_DC   PB12
#define TFT_RST  PB15
#define TFT_RGB_ORDER TFT_BGR
#define TFT_INVERSION_OFF      // fixes inverted colours on some ST7789 modules
#define LOAD_GLCD              // font 1 – frequency readout + splash
#define LOAD_FONT2             // font 2 – header + data grid
#define LOAD_FONT4             // font 4 – status bar, busy messages
#define SPI_FREQUENCY 4000000  // 40 MHz works; drop to 27 MHz for long wires
```

320x480 (ILI9488 / ILI9486):

```
#define ILI9488_DRIVER          // works for both ILI9488 and ILI9486
#define TFT_WIDTH  320
#define TFT_HEIGHT 480
// same TFT_MISO/MOSI/SCLK/CS/DC/RST/RGB_ORDER lines as above
#define LOAD_GLCD              // font 1 – splash credits
```

```
#define LOAD_FONT4           // font 4 – splash subtitle path
#define LOAD_GFXFF          // REQUIRED on this panel – free fonts
#define SPI_FREQUENCY 4000000 // don't skimp: this panel pushes 2.4x the pixels
```

Wire the panel: SCK→PA5, SDI→PA7, RES→PB15, D/C→PB12, CS→PB13. `TFT_MISO PA6` must be defined even if nothing is connected to it.

1.5 `build_opt.h` — leave it alone

The file contains three compiler flags and needs no action from you:

```
-DSERIAL_RX_BUFFER_SIZE=256 -DSERIAL_TX_BUFFER_SIZE=512
-DCDC_TRANSMIT_QUEUE_BUFFER_PACKET_NUMBER=16
```

The first line enlarges the serial buffers so GPS sentences are not dropped. The second grows the USB-CDC transmit queue from 128 bytes to 1 KB so the 1 Hz telemetry report (about 400 characters) always fits — without it, a host that connects but doesn't read could stall the report for seconds. The file is picked up automatically by the STM32 core. Do not delete it.

1.6 The Tools menu — board and runtime library

In Arduino IDE: Tools →

- **Board:** "Generic STM32F4 series" → **BlackPill F411CE**.
- **C Runtime Library:** **Newlib Nano + Float Printf/Scanf**. This one is mandatory. Without it the firmware compiles but prints garbage wherever a float should be (all the voltages, frequencies and phases).

1.7 Turn on USB CDC (so the board gives you a serial console over USB)

CDC is what makes `Serial` a virtual COM port on the USB cable. The boot banner, the command line, and the 1 Hz status report all travel through it. Without CDC the board still runs — the display works, the loop disciplines the oscillator — but over USB it looks completely dead: no banner, no prompt, nothing.

1. Tools → **USB support (or "USB support (if available)")** → **"CDC (generic Serial supersedes U(S)ART)"**. The exact entry name differs slightly between core versions; pick the one that mentions **CDC** and **generic Serial**.
2. Recompile and re-flash. Changing this setting changes the firmware — it is not a runtime switch.
3. After flashing, **press the board's RESET button** once. The chip re-enumerates from the DFU/upload USB device to the CDC serial device; without the reset some hosts keep talking to the old, now-gone device.
4. A new COM port appears (Windows: "STM32 Virtual COM Port", **VID 0483, PID 5740**). If Windows shows it as "Unknown device" or "device descriptor request failed", install the driver once with Zadig — see also Part 2.5.
5. Open the port at 115200. The firmware **waits up to 3 seconds** after reset for the host to open the port before printing the banner — so a slow driver does not swallow the first lines. If you open the terminal and see nothing, press RESET again with the terminal already open.

Two more things worth knowing:

- DFU (upload) mode and CDC (run) mode are **different USB devices** to Windows. The first few upload cycles may each trigger a driver installation round — that is normal, it settles.
- With `GPSDO_BLUETOOTH_PARALLEL` compiled in (the shipped default), everything USB shows is mirrored to the Bluetooth UART at **57600** — a ready-made fallback console if USB ever fights you.

1.8 Compile and check the size

Sketch → Verify/Compile. When it finishes, read the line `Sketch uses NNNNNN bytes`.

NNNNNN must stay below 393216. That is 384 KB — everything from there up belongs to the settings ring in sector 7 (Part 2). Ignore the percentage the IDE prints: it is calculated against the full 512 KB chip, so a build that

"fits" by percentage can still overrun into sector 7. v1.06 itself builds around 250 KB — plenty of headroom, just don't ignore a sudden jump.

Part 2 — Flashing, and the sector 7 rule

2.1 How the 512 KB flash chip is divided

Sectors	Address range	Contents
0 – 5	0x08000000 – 0x0803FFFF	Your firmware (384 KB)
6	0x08040000 – 0x0805FFFF	spare (growth buffer)
7	0x08060000 – 0x0807FFFF	the flash ring: settings + calibration + learned data

Sector 7 holds everything the device has learned about itself:

- your saved settings (algorithm choice, all loop parameters, timezone),
- the `CT` oscillator-sensitivity calibration,
- the `LC` phase-detector calibration,
- the learned drift/damping model and the last operating point.

Rebuilding all of that takes an hour of bench time. Protecting sector 7 is therefore **the single most important operational rule of this project**.

2.2 The golden rule

Normal uploads never touch sector 7. Full-chip erase destroys it.

- Arduino IDE upload, DFU, and the STM32duino bootloader reprogram only sectors 0–5. Your settings survive every routine firmware update.
- "Erase Chip" in ST-LINK Utility / STM32CubeProgrammer, or erase without an address range in J-Link, wipes the whole chip **including sector 7**. Never use it on a working, calibrated unit.

If you use ST-Link or J-Link tools to flash manually, erase **only** the firmware range and then load:

```
> erase 0x08000000 0x0803FFFF
> loadbin firmware.bin 0x08000000
```

If sector 7 ever does get wiped: nothing breaks. On the next boot the firmware detects a blank ring, formats it, and starts from defaults — you simply re-run `CT`, `LC`, set your algorithm and timezone, and `ES`. You lose the tuning, not the device.

Before the very first flash of a finished unit, make a full backup (J-Link example; ST-LINK Utility has the equivalent "save" button):

```
> JLinkExe -device STM32F411CE -if SWD -speed 4000
> savebin backup_full.bin 0x08000000 0x80000
> exit
```

To restore: `loadbin backup_full.bin 0x08000000`.

2.3 Flashing method A — ST-Link (simplest)

1. Connect ST-Link V2 to the BlackPill SWD pins (SWDIO, SWCLK, GND, 3V3).
2. Arduino IDE → Tools → **Upload method: "STM32CubeProgrammer (SWD)"**.
3. Press Upload. Done. The programmer only writes what the sketch occupies.

2.4 Flashing method B — DFU over USB (no programmer)

Which **BOOT0** do you have? Older WeAct BlackPills carry a **BOOT0** jumper; current boards (v3.1) have only a **BOOT0** push button. Use the matching recipe below.

Jumper boards: put the **BOOT0** jumper to **1**, press **RESET** — the board enumerates as "STM32 BOOTLOADER". After the upload, return the jumper to **0** and press **RESET**.

Button boards (v3.1), the reliable way: disconnect any external power; **press and hold BOOT0**, plug the USB cable in **keeping the button pressed**; after a few seconds release **BOOT0** — Windows should raise no "unrecognised device" alert and STM32CubeProgrammer sees the board. Upload, then press **RESET**. (Holding **BOOT0** while tapping **NRST**, as suggested elsewhere on the internet, usually does *not* work — especially with the board already mounted on a PCB.)

Windows may ask for a driver the first time — see the note below.

2.5 After any flash: the USB note

- After the IDE finishes, **press the board's RESET button** — the USB device re-enumerates cleanly from bootloader mode to the firmware's CDC serial.
- The firmware's USB serial is **VID 0483, PID 5740** ("STM32 Virtual COM Port"). If Windows refuses to open it, install the driver once with Zadig.

Part 3 — First start: calibration, algorithm, save

You need a terminal program (PuTTY, Terra Term, the Arduino IDE serial monitor, or the tuner from Part 8 — the tuner is the most comfortable).

3.1 Connect

- 1. Connect USB (and/or Bluetooth at 57600 — both work simultaneously in the shipped build).
- 2. Open the port at 115200, 8N1. Line endings: either CR or LF works.
- 3. Press **Enter**. You should see a prompt or the 1 Hz report start.
- 4. Type **H** and Enter. The full command list scrolls by — that is Part 7 of this manual, live.

The boot log, line by line. The first ten seconds after reset print a fixed sequence. Here is a typical one from a healthy, already-configured unit (exact lines depend on your hardware), followed by what every line means:

```
=====
GPSDO v1.06-rtos compiled 2026-08-23 14:32:45
FreeRTOS port by J. M. Niewinski with Claude, GLM-5.3 Max & Qwen3.8-Max AI
https://github.com/jmnlabs/GPSDO_FreeRTOS
Inspired by GPSDO v0.06c by Andre Balsa
https://github.com/AndrewBCN/STM32-GPSDO
Algos 0-2 original design by Andre Balsa
Algos 3-9 by J. M. Niewinski
Algo 10 (LTIC 3-stage) inspired by Dan Wiering's measurements
Algo 11 (LTIC-Lars) after Lars Walenius' PI loop
Algo 12 (multi-level accumulator) after Alan Cashin (MIS42N)
Type H = help SW = stack diagnostics
=====
Reset cause: POWER-ON/BROWN-OUT PIN/NRST
Flash ring: sector 7 ready
Settings: recalled from flash ring
Live store: LRN + LC applied from flash ring
Initial PWM=44653 algo=12 time_offset_min=120
GPS init: probing baud rate...
GPS detected at 38400
GPS: disabling noisy NMEA at 38400 baud
GPS: 4/4 NMEA sentences disabled
GPS: sending UBX config at 38400 baud
UBX: CFG-NAV5 ACK
LEA-T: accepted CFG-TMODE2 (28B)
Hardware configured, creating RTOS objects...
Timers started
Starting FreeRTOS scheduler
HW: AHT10/AHT20 sensor OK (I2C 0x38)
HW: BMP280 sensor OK (I2C 0x77)
HW: INA219 sensor OK (I2C 0x40)
HW: LTIC phase input OK (PA1 analog)
TFT: init start (SPI1 PA5/PA7, CS=PB13 DC=PB12 RST=PB15)
TFT: freq-band sprite (4-bit) created
TFT: header sprite (4-bit) created
TFT: data sprite (1-bit) created
```

After that the 1 Hz report starts (Part 6). What each line tells you:

Line	Meaning
banner (GPSDO v1.06-rtos ...)	firmware identity. If you see garbage characters instead: wrong baud rate — use 115200
Reset cause:	why the chip restarted. POWER-ON/BROWN-OUT PIN/NRST = normal power-up or reset button. SOFTWARE = a reset commanded by firmware (RB, end of an upload). INDEP-WDG/WINDOW-WDG = a watchdog (this firmware runs none — treat as a fault signal). The extra line -> supply dipped: check the 3V3 rail under load appears after a brown-out: your 3V3 supply sagged under the OXCO load — fix the power supply, don't ignore it

Line	Meaning
Flash ring: sector 7 ready	the settings ring in sector 7 was found valid
Flash ring: sector 7 blank/formatted (defaults)	virgin or wiped ring — normal on the very first flash ; the firmware formats it and uses compile-time defaults
Settings: recalled from flash ring	your saved settings (algorithm, loop params, timezone, flags) were applied
Settings: none stored (compile-time defaults)	nothing saved yet — expected on a new unit; run through Part 3 and ES
Live store: LRN + LC applied from flash ring	learned data (LC calibration, drift model, last operating point) applied — this line only appears once you have a calibrated unit
Initial PWM=... algo=... time_offset_min=...	the operating point chosen at boot: final PWM code (live data overrides settings when fresher), the restored algorithm, and your timezone offset in minutes
GPS init: probing baud rate... / GPS detected at ...	the receiver is found and its baud measured (it prefers 38400)
GPS: no response to baud probe, defaulting to 9600	no receiver answered — check wiring to the GPS module before anything else
GPS: disabling noisy NMEA... / GPS: sending UBX config... / UBX: CFG-NAV5 ACK	the receiver is switched to binary mode and stationary dynamics; NAK/no response variants are survivable — the receiver just keeps its current configuration
LEA-T: accepted CFG-TMODE2	a timing receiver (LEA-M8T class) was configured into survey-in / Time Mode — appears only with GPSDO_GPS_TIMING and a timing receiver
Hardware configured... / Timers started / Starting FreeRTOS scheduler	internal bring-up; all three should always appear
HW: <sensor> OK (I2C ...) / HW: <sensor> not found	the I2C bus scan: each sensor reports present or not. A "not found" sensor is not fatal — the unit runs without it, that field just stays empty in reports
HW: LTIC phase input OK (PA1 analog)	the phase detector input is configured (algorithms 10–12 depend on it)
TFT: init start (...)	display bring-up begins. If nothing follows this line , the TFT wiring or User_Setup.h is wrong (Part 1.4) — that is the "white/dead screen over serial is fine" case
TFT: ... sprite FAILED – direct-draw fallback	the display works but with slower redraw (out of RAM for the flicker-free sprites) — cosmetic, not fatal

Lines that appear **later**, while the unit runs, and what they mean:

Line	Meaning
LTIC: running UNCALIBRATED (run LC)	an algorithm 10/11/12 is active but LC has never run — the phase numbers are nominal, not measured. Run LC

Line	Meaning
LTIC ACQ: polarity unset – run 'LPOL -1' (or +1)	algorithm 10 refuses to steer until you set the polarity (it holds safely instead of guessing). Algorithms 11 and 12 used to assume +1 and steer anyway; they now hold the same way
picDIV: armed (output stopped, waiting for 1PPS sync)	the divider was armed; a 1–1.2 s gap in its output is expected before it syncs to the next PPS edge
LEA-T: ... survey ... % / s) – continuing anyway	survey-in progress monitor; "continuing anyway" means the survey hit the time limit above the accuracy target — usually a blocked sky view
!!! FreeRTOS ... (configASSERT / STACK OVERFLOW / MALLOC FAILED)	a firmware fault was caught, with file/line or task name — the blue LED blinks rapidly at the same time. Note both down; the unit needs a reset (RB)

A virgin unit automatically requests a calibration on the first boot — you may see the calibration countdown before anything else.

3.2 Let the GPS settle

- Give the antenna a clear view of the sky. A window sill works; a geodetic antenna on a roof works better.
- With a timing receiver (M8T/F9T...) and `GPSDO_GPS_TIMING` compiled in, **survey-in** runs at boot: the receiver measures its own position for at least 300 s until the estimate is better than 5 m, then switches to fixed-position **Time Mode**. From then on the display shows `HDOP:TIME` instead of a number. Survey-in needs to complete only once; it re-runs after power loss. (`SV 0` disables it, `SV 1` re-enables, next boot.)
- The OCXO warm-up takes 300 s (`WU 0` skips it; leave it on).
- The yellow LED is **off without a GPS fix, steady on with a fix** — glance there first when something looks wrong.

Which GNSS module? A genuine u-blox timing receiver (LEA-M8T class) is what this firmware is built around: survey-in, Time Mode, the `qErr` sawtooth correction. It will also discipline from cheap navigation modules — including the Chinese u-blox clones common on eBay/AliExpress — because the 1PPS pulse and the NMEA sentences are all the loop strictly needs. But clones ignore the binary configuration: expect `0/4 NMEA sentences disabled`, unacknowledged CFG frames and a ~15 s slower boot while the timeouts expire; there is no survey-in and no `qErr`, the display shows a numeric HDOP forever, and the extra position wander lands on the phase — the loose default algo-12 limits absorb exactly that. One known clone quirk: after the failed configuration round some modules stop sending data and the display stays on "acquiring" — the cure is to enter the u-center tunnel (`T` with the module's baud, e.g. `T 9600`) and simply let it time out; the re-probe that follows restores operation. A firmware switch covers all of it: define `GPSDO_FAKE_UBLOX` in `gpsdo_config.h` and the firmware probes the baud rate and sends nothing else — one switch, regardless of any other GPS options enabled. Not yet hardware-tested (no clone module on the author's bench); field reports welcome.

3.3 Calibrate — `CT` first, then `LC`. The order matters.

These two commands teach the firmware two different physical facts, and the second needs the first:

Step 1 — `CT` (oscillator sensitivity + loop tuning, ~3 minutes). Type `CT` and Enter. The firmware drives the control voltage to three points (1.5 V, 2.0 V, 2.5 V), measures the frequency at each, fits a straight line, and computes **K** — **how many hertz one PWM step is worth on *your* oscillator** (accepted range 0.02–2 mHz/LSB). From K it then derives sensible PID gains for algorithms 3–9 and the LTIC loops, and **auto-saves the PID group** to flash. Do not power off during those three minutes.

Step 2 — `LC` (phase detector calibration, ~5 minutes). Type `LC` and Enter. The firmware arms the picDIV divider, centres the detector, sweeps one capacitor ramp, and measures the detector's **ns per volt, zero-offset volts and range in ns**. On PASS it **auto-saves**. It refuses to be useful if `CT` hasn't run — that is why the order matters.

Sanity check: type `LL` and confirm the LTIC block shows non-zero `ns_per_volt`, `zero_offset`, `range_ns`. Type `DAC` and it will even tell you the size of one output step in microhertz.

3.4 Choose the algorithm

- **LA 12** — the multi-level accumulator (after Alan Cashin). The flagship of v1.06: holds phase to a few ns RMS, corrects on average every few minutes. **This is the recommendation.**
- **LA 11** — Lars Walenius' continuous PI loop. The mature, gentle classic; excellent long-term behaviour with one knob (LTC).
- **LA 10** — three-stage phase loop (ACQ→DPLL→LOCK). Solid, more parameters.
- Algorithms 0–9 are the historical collection — they work, they are frozen, see Part 4.

LA alone shows the current algorithm. Selecting does **not** save — see step 3.6.

3.5 Recommended extras

- **SAW 1** — sawtooth correction. The GPS receiver's 1PPS quantisation error ($\pm 8... \pm 25$ ns, reported every second as **qErr**) is subtracted from the measured phase. With a timing receiver this is free accuracy; turn it on.
- **TZ <city>** — local time with DST (e.g. **TZ Warsaw**, **TZ Adelaide**), or **TO 1** for a fixed offset, **LT 1** to show local instead of UTC. **H TZ** explains the rule format if your zone is missing.
- **PO <Pa>** — offset added to the BMP280 pressure reading (–5000..5000).
- **AO <m>** — offset added to the **GPS altitude** at display/telemetry (–3000..3000 m). This corrects the displayed altitude to your real elevation, e.g. if the geoid model puts you 30 m off: **AO 30**.

3.6 Save everything — ES

Type **ES** and Enter. This writes **all** settings plus the live (learned) data to the flash ring in sector 7. From this moment a power cut loses nothing.

A handful of preference commands save **themselves** the moment you set them and tell you so — the reply says **[auto-saved: ...]**. These are: **TZ**, **TO**, **LT** (timezone group), **WU**, **SPL**, **SV** (flags), **PO**, **AO** (offsets), plus **CT** (auto-saves the PID group it just tuned) and a passing **LC** (saves itself to the live ring).

Everything **loop-related** — gains (**KP/KI/KD/IL**), all LTIC and Lars parameters, the algo-12 settings, and the **LA** algorithm selection itself — lives in RAM only until you save it. Every such command answers with a hint like **[not saved – run 'ES LTIC' to keep it]**: run the named group save, or just remember: **after a tuning session, ES**.

3.7 What "healthy" looks like

- The trend word (display + **PWM:** report line) reaches **LOCK** and stays there — with algo 12, brief **CORR/ZC** flashes are *normal and healthy* (a correction is the algorithm self-testing on schedule; the display still counts them as locked).
- The phase (**dph:** in the report) wanders around zero within tens of ns and always comes back — it must not ramp away.
- The frequency windows close in: **100s:** in low millihertz, **1ks:** in microhertz, over an hour or two.
- **CS** (correction statistics) prints small, steady RMS numbers that don't grow hour over hour.
- On the TFT the big frequency number is **green**.

Expect, on a quiet bench with algo 12: phase 5–20 ns RMS, corrections of a few LSB every few minutes, Allan deviation in the 1e-11...1e-12 class from 1000 s upward. Near a heater, an openable door, or direct sun the numbers will be worse — that is physics, not a bug.

Part 4 — The thirteen algorithms (0–12) and their parameters

One idea underlies everything: the firmware counts the OCXO frequency with a gated counter (TIM2), measures the GPS-vs-OCXO phase with the LTIC detector, and adjusts the control voltage (PWM "DAC", 65536 steps, $\sim 48.8 \mu\text{V}$ per step at 16 bit; with `GPSDO_PWM_DITHER` the effective resolution is 24 bit — about $0.2 \mu\text{V}$ per step). Sign convention: measured error $e = \text{freq} - 10 \text{ MHz}$; if $e > 0$ the oscillator is fast and the PWM must go **down** (for a positive-sensitivity EFC; `LPOL` / polarity handling covers inverted EFCs).

If phase, frequency error, or PID are new words to you, stop here and read Appendix A (what the loop is chasing and why it must be gentle) and Appendix B (what the letters P, I and D actually do). Ten minutes there will make every parameter below readable.

The `Learn:` line in the report names the active algorithm; the tuner picks its plot family from it automatically.

4.0 The menu

#	Name (as shown)	One-liner	Status
0	primitive	André Balsa's original stepped controller, 429 s cycle	default after cold flash
1	forced-drift	+1 LSB per 1000 s — oscillator characterisation	diagnostic
2	random-walk	± 1 LSB noise every 5 s — noise-floor measurement	diagnostic
3	FLL-PID-man	PID on the 100 s frequency average	classic
4	PLL-PI-man	PI on phase, 10 s cycle	classic
5	PLL-PID-man	as 4 with its own tuning slot	classic
6	FLL-PID-gen	FLL PID, genetically-derived tuning	classic
7	PLL-PID-gen	the old workhorse PLL; its Kp stores the CT result	classic
8	hybrid-FLL-PLL	sigmoid blend of 6 and 7 by error size	classic
9	NN-MLP	small neural net + learned temperature steering in holdover	classic
10	LTIC-3stage	ACQ→DPLL→LOCK state machine on the phase detector	recommended line
11	LTIC-Lars	Lars Walenius' continuous PI on phase	recommended line
12	multi-level	Alan Cashin's multi-level accumulator	flagship

Algorithms 0–9 are **frozen**: they remain because they work and because algorithm 7's tuning slot doubles as the storage of the CT calibration. New development is 10/11/12 only.

Select with `LA <n>` (persists via `ES ALGO`). A fresh unit always boots algorithm 0 — select and save yours once, and it sticks forever after.

4.1 Algorithms 0–2 (diagnostic)

No parameters. 0 steps the PWM when frequency averages cross fixed thresholds. 1 ramps the PWM linearly (characterise the EFC). 2 injects noise (measure the loop's floor). You will not need them for normal operation.

4.2 Algorithms 3–9 (the classics) — parameters via KP/KI/KD/IL

`KP <algo> <val>` / `KI` / `KD` set the gains (algos 3–7, value 0..100000); `IL <algo> <val>` sets the integrator clamp (algos 3–9, 100..100000). `LP [n]` lists. Defaults (also what CT derives from, then overwrites with measured-based values):

algo	Kp	Ki	Kd	I_LIMIT
3	70.0	0.70	175.0	9000

algo	Kp	Ki	Kd	I_LIMIT
4	1000	0.020	2.0	7000
5	1000	0.020	2.0	10000
6	205	0.264	14950	13000
7	1000	0.020	2.0	10000
8	—	—	—	13000
9	—	—	—	450

Extras: algorithm 8 has **BC** (blend crossover, default 0.024 Hz) and **BS** (blend scale, 0.012 Hz) — the sigmoid centre and width that decide how much FLL vs PLL is mixed in at a given error; algorithm 9 has **NS** (max step, default 175 LSB). Save any of this with **ES** PID. Honestly: after **CT** you should not need to touch these — that is what CT is for.

4.3 Algorithm 10 — LTIC three-stage

A state machine on the phase detector: **ACQ** (frequency-led pull-in, also re-centres the detector ramp) → **DPLL** (2 s cycle, frequency + phase) → **LOCK** (corrections every **LIV** seconds using a two-window pair estimate).

Parameters (all **ES** LTIC; **LL** lists everything):

Command	Meaning	Range / default
LAT	ACQ phase threshold	ns, default 100
LDT	DPLL→LOCK drift threshold	1e-13..1.0, default 5e-10
LIV	LOCK correction interval	1..600 s, default 300
AQP/AQI/AQD/AQL	ACQ-stage PID	0..100000
DPP/DPI/DPD/DPL	DPLL-stage PID	0..100000
LKP/LKI/LKD/LKL	LOCK-stage PID	0..100000
LPOL	PWM→phase polarity, -1/0/+1	0 = auto
LCV	ACQ centring target volts	0 (= auto mid-range) .. 3.3
ACG g [cap]	centring drive gain [max step]	50..20000 LSB/V [5..1000 LSB]
FA/FAD/FAL	damping frequency-average window (both / DPLL / LOCK)	10, 100 or 1000 s

ltic_autotune (part of CT/LC) fills the stage gains from the measured sensitivity, so the shipped sequence already tunes this algorithm. If ACQ never leaves, check **LPOL** — the loop warns and holds until you set it.

4.4 Algorithm 11 — LTIC-Lars continuous PI

Lars Walenius' loop, ported faithfully. One PI controller evaluated every second, with an adaptive pre-filter; lock when the filtered phase stays inside **LPL** ns for **LTC** × **LPF** seconds. Railed detector → automatic frequency-led pull-in, one picDIV re-arm allowed.

Command	Meaning	Range / default
LG	loop gain. 0 = auto from CT (recommended)	0..10000
LD	damping	>0..1000, default 3
LTC	loop time constant — the one knob	1..600 s, default 60
LFD	pre-filter divisor (pre-filter = LTC/this)	1..100, default 2
LTO	phase target, detector ADC counts	0..4095, default 2620
LPL	lock phase window	1..10000 ns, default 100

Command	Meaning	Range / default
LPF	lock hold factor (hold = LPF × LTC)	1..100, default 5
LTK	temperature feed-forward, DAC steps per ADC step	−32000..32000, 0 = off
LTR	temperature reference, ADC counts	0..4095

Tuning guidance in one sentence: leave **LG 0**, set **LTC** to how gently you want the loop to steer (60 s is a good start; 240 s calms a noisy site; short only makes it noisier), and let the rest default. All **ES LTIC**. The tuner's **LTIC-Lars** tab exposes exactly these fields.

4.5 Algorithm 12 — the multi-level accumulator (flagship)

Ported from Alan Cashin's (MIS42N) PIC GPSDO and refined during v1.04–v1.06. No fixed cycle time at all: **the size of the error chooses the averaging time**.

How it works, in plain words. Every second the phase reading enters a ladder of accumulators. Level n averages over $2^{(n+1)}$ seconds — the ladder runs 2, 4, 8, ... 2048 s (11 levels). Each level holds a pair of values (A = older half, B = newer half). When a pair completes, two numbers are computed: the **slope** ($B - A$ = frequency error over that span) and the **extrapolated phase** ($3B - A$ = phase right now). If the phase stays inside that level's limit, the pair is summed and promoted one level up (twice the averaging). If it exceeds the limit, a correction fires immediately, the ladder resets, and the correction is spread gently over the span it came from (minimum 64 s — "a GPSDO wants a large error corrected gently"; Alan's own floor is 16 s, chosen so that a warm-up correction cannot demand a control voltage outside the DAC's range — a smaller floor means bigger steps, a larger one means slower phase recovery). Independently, reaching level **MR** (default 9 = 1024 s) forces a correction even under every limit — otherwise a slow drift would never be acted on.

Each correction has up to three parts, and this is deliberate: cancel the measured **frequency** error, move the **phase** back to zero, and never one without the other. Then a trick Alan calls essential: the deliberate slew is remembered, and on the first phase reading that crosses zero in the expected direction, exactly that slew is removed (**ZC**, zero-crossing cancellation) — leaving the oscillator at the right frequency *and* no phase error.

Trends specific to 12: **WAIT** (no data yet) → **SYNC** (5 s settle after the divider arms) → **FLL** (detector railed, frequency pull-in) → **NOPH** (no valid phase, PWM frozen) → **ACQ** → **LOCK** (>16 s of genuine quiet *and* frequency home; corrections and **ZC** do *not* reset the quiet timer — they are health, not noise). Transient **CORR** / **ZC** flashes are normal. **NoCT** means: gain is auto but **CT** has never run — run **CT**. **NoPL** means the EFC polarity is unset — the loop holds until you set **LPOL -1** or **1**.

Parameters (**ES ALG012**; **ML** lists everything including the live 1-sigma noise estimate):

Command	Meaning	Range / default
MG	gain, LSB per ns of phase. 0 = auto from CT	0..10000, default 0 (auto)
MR	level at which a correction is forced	0..10, default 9 (1024 s)
MF	where the per-level limits come from: 0=follow MG, 1=stored table, 2=sigma formula, 3=measured fit	0..3, default 0
MFT	with MF 3: target seconds between noise-driven corrections	0 (=3600) or 2048..65535 s
MLP n v	set one row of the 11-level limit table	level 0..10, value 1..500000 accumulator units

The default limit table is Alan's own (scaled from his 25 ns detector to this ~1 ns one). In nanoseconds per span it is: L0 462, L1 400, L2 331, L3 264, L4 191, L5 164, L6 126, L7 117, L8 108, L9 103, L10 63 ns. **ML** labels it **UNTUNED** (only the 128 s row was ever derived from a spec) — treat it as the proven starting point, not scripture. Its provenance, in Alan's own words, is a **worst-case reception scenario**: the numbers were calculated for a NEO-6 with an indoor antenna, whose 1PPS can be out by ±100 ns for short stretches (±30–40 ns on average). The table is loose on purpose — sized for the worst receiver the design might face, not for the LEA-M8T-class hardware this build usually has.

Which MF should you use? Field results from two boards:

- **Quiet site (indoor window antenna, stable temperature):** the sigma formula (MF 2) or measured (MF 3) works — the loop self-scales and corrects every few minutes.
- **Noisy site (workshop with a door, heater nearby, moving air):** the auto-scaling can hunt at level 0 (limits tighter than the environment's real wander). There, Alan's loose stored table (MF 1) was dramatically more stable. If you see constant level-0 corrections, switch to MF 1.

Behind both results sits Alan's stated design intent, worth knowing before you tune: *"we don't want the algorithm using limits, we want the algorithm using scheduled corrections at defined intervals."* The run-level correction (MR) is meant to be the workhorse and the limits only a safety net for warm-up and reception excursions — which is exactly why his table is loose. A tight table firing constantly is, by that philosophy, working against the algorithm rather than with it.

The gain (MG) belongs to the **oscillator**; the limits belong to the **site noise** — that is exactly why the two are set by different commands and why MF lets you mix a measured gain with a hand-set table.

In the tuner: the **Multi-level (algo 12)** tab has the four scalars and the whole 11-row limit table with Send/Read/List buttons, and the live plots switch to the algo-12 family (ph phase error, level, correction count, sigma, ZC count) automatically.

Part 5 — Displays: what every field means

Several displays can run at once (I2C + SPI don't conflict). All show the same truth, at different levels of detail.

5.1 The TFT (480x320 or 320x240)

```
y= 0..23   header bar: "GPSD0 v1.06"           LMT 14:32:45 Thu
y= 30..62   FREQUENCY — big digits, colour-coded
y= 70..151  info grid, two columns
y=156..195  sensor row
y=204..239  status bar
```

The big frequency number's colour is the one-glance health check:

Colour	Meaning
green	locked (for 10/11: trend LOCK; for 12 also during CORR/ZC — a correcting loop is a healthy loop)
white	adjusting
orange	holdover
red	no signal / no fix

While boot procedures run, the number is replaced by orange countdowns: Survey <s> ±<m>, OCX0 warmup <s>, Tune <s> (CT), LTIC cal <s> (LC), Calibrate <s>.

Info grid: left column — UTC time + weekday, date, uptime (counted from the GPS 1PPS, so it doesn't drift), Algo: n <trend>, PWM: code + Vct: control voltage. Right column — Sat: count + HDOP: (or HDOP: TIME after survey-in — that word is *good*, it means fixed-position time mode), Lat:/Lon: (6 decimals), Alt: + qErr: (the sawtooth value being subtracted), INA: supply volts and current. Sensor row: BMP: temperature + pressure, AHT: temperature + humidity, Vph:/dph: detector voltage and derived phase in ns (ovf = ramp outside its valid band), Vcc:/Vdd: rails.

Status bar (colour-coded background): green DISCIPLINED FIX OK, orange HOLDOVER (manual), red HOLDOVER (fix lost) or WAITING FOR GPS FIX; SURVEY appended while survey-in runs.

5.2 The OLED (128x64) — two pages, flipping every 10 s

Page A (GPS): local time, frequency, Lat/Lon/Alt+Sats, uptime, UTC+AHT temperature, PWM + trend (a blinking H/A at the right edge = manual/auto holdover). Page B (sensors): BMP/AHT/INA rows, Sat+HDOP, UTC. During procedures the frequency row shows F SVIN <s>s <m>m, F WARMUP <s>s, F CAL <s>s instead.

5.3 The 20x4 LCD

Line 0 frequency; line 1 UTC + uptime days; line 2 rotates every 10 s (coordinates / sats+HDOP / AHT / INA / BMP); line 3 PWM + Vctl + trend with blinking holdover marker.

5.4 The LED clock digits (TM1637 / HT16K33)

Local time HH:MM (colons blink on the second). All dashes = boot/no data; 0000 = no fix; spinners = warm-up / survey / calibration.

5.5 LEDs on the board

LED	Meaning
Yellow (PB8)	OFF = no GPS fix · steady = fix OK · slow blink (1 s) = manual holdover · fast blink (200 ms) = fix lost, auto holdover
Blue (PC13)	fault only — rapid blinking means a firmware fault was caught (assert / stack overflow / heap failure), with the reason on serial. It is <i>not</i> a heartbeat; a dark blue LED is the good state.

Part 6 — Serial telemetry (the 6-line report)

Once per second (once per PPS) the firmware prints a status block, in this shape (human-readable mode, `RH`):

```
Up: 000d 02:15:33 UTC: 22/8/2026 14:32:45
Lat: 51.477928 Lon: -0.001531 Alt: 46.5m Sat:10 HDOP:TIME
Freq: 1000000.0000 Hz 10s:0.0 100s:0.02 1ks:0.000 10ks:0.0000
PWM:44653 Vctl:1.970V hit
Learn: algo=11 (LTIC-Lars) gain=auto scale=46 phase=12.3ns LOCK qErr=-8.2ns
BMP:23.4C 1013.2hPa AHT:22.1C 45.3%RH INA:12.05V 250mA Vphase:3.077V dph:1390.5ns
```

(The example position is the Royal Observatory, Greenwich — longitude zero by definition, and a deliberately public place. Your unit will of course print its own coordinates; if you share logs, remember the tuner's **Redact position** option.)

Line by line:

Line	Fields
1	uptime (PPS-counted, trustworthy from v1.06) + GPS UTC date/time
2	position (6 decimals), altitude (GPS altitude plus your AO), satellites, HDOP — or the word <code>TIME</code> when the timing receiver is in fixed-position mode; no fix → GPS: no position fix yet
3	raw counted frequency (or --- before the first count) and the averaged error windows 10 s / 100 s / 1 ks / 10 ks — each appears only once its buffer has filled
4	16-bit PWM code, measured control voltage, trend word (<code>hit</code> , <code>ACQ</code> , <code>DPLL</code> , <code>LOCK</code> , <code>PLL</code> , <code>CORR</code> , <code>ZC</code> , <code>NOPH</code> , <code>NoCT</code> , ...) — in holdover, <code>[HOLDOVER]</code> replaces the trend
5	algorithm-specific learn line (see below) + <code>qErr</code> when <code>SAW 1</code> is active
6	BMP280 temperature/pressure (raw + your PO), AHT temperature/humidity, INA voltage/current, LTIC detector voltage <code>Vphase</code> , derived phase <code>dph</code> in ns (sawtooth-subtracted like the loop itself)

The Learn line per family:

- algo 11: `gain=auto|<val> scale=<n> phase=<ns> LOCK|acq`
- algo 12: `ph=<ns> level=<n> corr=<n> arm=<n> sig=<ns> zc=<n> secs=<n>` — accumulated phase, last acting level, corrections, arms, live noise estimate, zero-crossings, seconds since start
- algo 10: `state=ACQ|DPLL|LOCK`
- algos 3–9: learned drift/slope/damping, algo 9 adds the learned `tempco`

Tab-delimited mode (`RD`) prints one line per second with the same data as columns (uptime, frequency windows, sats, HDOP, PWM, voltages, all sensors, raw TIC) — the format choice for logging into a spreadsheet. `RP/RR` pause/resume. The report is non-blocking: if a host connects but doesn't read, the firmware drops report tails rather than freeze the displays (fixed in v1.06 — a full CDC queue used to wedge the display task for good).

Part 7 — Command reference (every command)

Serial, 115200, case-insensitive, Enter-terminated. Commands with a `[val]` argument **show the current value when called with no argument**. Two saving regimes exist — the firmware always tells you in its reply which one you just hit:

- **Preferences save themselves** and print `[auto-saved: ...]: TZ, TO, LT, WU, SPL, SV, PO, AO` — plus `CT` (auto-saves the PID group it derives) and `LC` (auto-saves on PASS).
- **Loop parameters live in RAM until you save them** — everything the tuner edits: gains, LTIC/Lars/algo-12 parameters, `LA`, `SP`. The reply names the exact group to keep the change (`[not saved – run 'ES ...']`).

Version, help, state

Command	What it does
V	version, authors, credits
H / ?	command list · H TZ = timezone details
SW	stack watermarks, free heap, uptime + its source, MCU-vs-GPS ppm

Output (the DAC)

Command	Range	What it does
SP [n]	1..65535 (no arg = 32767, mid-scale ≈ 1.65 V)	set the control DAC directly — manual control during experiments
up1/up10/dp1/dp10	—	nudge PWM $\pm 1/\pm 10$ (refused during a running calibration)
DAC	—	report: output path, 24-bit and 16-bit codes, measured Vctl, one step in μHz (needs CT)

Calibration

Command	Duration	What it does
C	~2 min	older 2-point PWM centring
CT	~3 min	oscillator sensitivity K + auto-tunes PID 3–9 and LTIC; auto-saves PID — run this first
LC	~5 min	phase-detector calibration (ns/V, offset, range); auto-saves on PASS — run this second
ACG g [cap]	—	ACQ centring drive: gain 50..20000 LSB/V, max step 5..1000 LSB
AP	—	arm the picDIV divider manually

Mode and reporting

Command	What it does
RH / RD	human-readable / tab-delimited report
RP / RR	pause / resume the 1 Hz report
MH / MD	holdover (freeze PWM, fly alone) / disciplined
F	flush the frequency averaging ring buffers
T [baud]	transparent GPS tunnel on USB for u-center, 300 s (baud 4800..921600)

Algorithm selection and classic PID (see Part 4)

Command	Range	What it does
LA [n]	0..12	select / show the loop algorithm
LP [n]	algo 0..9	list PID parameters
KP/KI/KD n val	algo 3..7, 0..100000	set gains
IL n val	algo 3..9, 100..100000	set integrator clamp
BC / BS	0.0001..1.0 Hz	algo 8 blend crossover / scale
NS	1..10000 LSB	algo 9 max step

LTIC (algo 10) — save with ES LTIC, list with LL

Command	Range / default	Meaning
LNV	0..1e6	detector slope, ns per volt (LC measures it)
LZO	0..3.3 V	detector zero-offset volts
LRN	0..1e9	detector range, ns — <i>note: this command sets the detector range; the "self-learning on/off" meaning listed in the help is unreachable in v1.06 (see quirks below)</i>
LAT	0.001..1e9 ns (100)	ACQ phase threshold
LDT	1e-13..1.0 (5e-10)	DPLL→LOCK drift threshold
LIV	1..600 s (300)	LOCK correction interval
AQP/AQI/AQD/AQL, DPP/DPI/DPD/DPL, LKP/LKI/LKD/LKL	0..100000	stage PIDs
LPOL	-1 / 0 / +1	PWM→phase polarity (0 = auto)
LCV	0..3.3 V	ACQ centring target
FA/FAD/FAL	10 / 100 / 1000 s	damping average window (both / DPLL / LOCK)

LTIC-Lars (algo 11) — save with ES LTIC

LG (0..10000, 0=auto), LD (default 3), LTC (1..600 s, default 60), LFD (1..100, default 2), LTO (0..4095 ADC, default 2620), LPL (1..10000 ns, default 100), LPF (1..100, default 5), LTK (±32000, 0=off), LTR (0..4095) — meanings in Part 4.4.

Algo 12 (multi-level) — save with ES ALG012, list with ML

MG (0..10000 LSB/ns, 0=auto from CT), MR (0..10, default 9), MF (0.3 limits source, default 0), MFT (0=3600 s, or 2048..65535 s), MLP <level> <units> (level 0..10, value 1..500000) — meanings in Part 4.5.

GPS, time, sensors

Command	Range	What it does
SV 0 1	—	survey-in / Time Mode off/on (applies at next boot) — auto-saved
TZ <city> <rule>	e.g. TZ Adelaide	timezone with DST (H TZ for details) — auto-saved

Command	Range	What it does
T0 <h[:mm]\ A>	-14..+14	fixed UTC offset, or A = auto from GPS position (EU DST rule) — auto-saved
LT 0 1	—	show UTC / local time — auto-saved
P0 <f>	-5000..5000 Pa	pressure offset added to the BMP280 reading — auto-saved
A0 <f>	-3000..3000 m	altitude offset added to the GPS altitude — auto-saved
SAW 0 1	—	sawtooth (qErr) correction off/on — bare SAW shows status; recommended ON with a timing receiver (save with ES FLAGS)
WU 0 1	—	OCXO warm-up on boot — auto-saved
SPL 0 1	—	boot animation on/off — auto-saved

Saving, recalling, resetting

Command	What it does
ES [obj]	save everything (no argument) or one group: TZ, PID, LTIC, FLAGS, ALGO12, ALGO, P0
ER	recall — re-read settings from flash now (undo unsaved changes)
EE	erase the settings slot — defaults on next boot
EW	flash-ring wear: erase cycles, slots used, sector/address
FR	read-only ring status (always on since v0.96)
CS	correction statistics: count, peak, RMS over the last 100/1k/10k/100k corrections — small and steady is good, growing is bad
RB	warm reboot (keeps settings — but does not auto-save first)
CR YES	cold restart: wipe settings + learned data, factory defaults (the YES is required because the learned model takes days to rebuild)

Known quirks in v1.06 (honest list)

1. **MZ** appears in the help but has no handler — zero-crossing cancellation is always on in code; there is nothing to toggle.
2. **LRN** is shadowed: the LTIC float table claims it first, so **LRN** sets the detector *range in ns*. The **LRN 0|1|R** self-learning toggle shown in the help cannot be reached in this build.
3. **FR 0|1** in the firmware help is stale — the ring is always on; **FR** ignores arguments.
4. **H CS** mentioned by the **CS** output doesn't exist as a sub-page; **H** alone prints the whole list.

Part 8 — The PC tuner

`tools/gpsdo_tuner.py` — a desktop console for watching and tuning. It does everything a terminal does, plus live plots and CSV logging.

8.1 Install and start

Python 3.9 or newer, then:

```
pip install PySide6 pyqtgraph pyserial tzdata
```

(`tzdata` is only used by the *Generate `tz_table.h`* button; on Linux/macOS the system provides it.)

Run: `python gpsdo_tuner.py` (on Windows you can double-click it).

8.2 Connect

Pick the port in the toolbar, baud 115200, Connect. The tuner immediately reads the firmware version and **all** current parameters (`LL`, `FA`, `LP 3–LP 9`, every Lars and algo-12 verb, `ML`) and fills every tab — you see the device's real state, not defaults. The state line shows `state: LOCK (locked)` etc.; plot titles follow the active algorithm.

8.3 The tabs

Tab	What it edits
LTIC (algo 10)	the three stage-PID quartets; Read (<code>LL</code>) / Save (<code>ES LTIC</code>) / Revert (<code>ER</code>)
LTIC-Lars (algo 11)	<code>LG LD LTC LFD LTO LPL LPF LTK LTR</code> with Set buttons
Multi-level (algo 12)	<code>MG MR MF MFT</code> + the whole 11-row limit table (Send limits, Read all, List (<code>ML</code>), Save (<code>ES ALG012</code>))
FA damping	DPLL / LOCK damping windows
PID algo 3-9	<code>Kp/Ki/Kd/IL</code> per algorithm
Calibration	<code>LNv LZ0 LRN LCV LAT LIV LPOL</code>
Raw monitor	everything the board says, unparsed; logging controls
Help	the firmware command reference

Remember the firmware rule applies here too: **Set buttons change RAM only; the tab's Save button runs the `ES ...` that persists.**

8.4 The plots

Three panes, refreshed continuously from the 1 Hz telemetry. Top panes depend on the algorithm family — phase (`dph/ph`) + detector voltage for 10/11/12 (with grey anchor and red valid-band guide lines once calibration is known), drift + control voltage for the classics. The bottom pane is always the frequency error in Hz. Toolbar: time-span selector (1 min ... all), Follow live, Clear. Resolution is the telemetry rate — events faster than ~2 s are invisible, and the plots trust the board.

8.5 Logging (the plots are not a logger)

Raw monitor → **Start logging**, format Full log / CSV only / Both:

- **Full log** `gpsdo_YYYY-MM-DD_HH-MM-SS.log` — every line as printed, ~217 MB per week at 1 Hz.
- **CSV** — parsed columns: `utc`, `up_s`, `algo`, `state`, `dph_ns`, `qerr_ns`, `vphase_v`, `pwm`, `f10`, `f100`, `ph_ns`, `level`, `corr`, `sig_ns`, `zc`, `bmp_c`, `sat`, `hdop` (~65 MB/week). Empty cell = field absent that second (never zero).
- **Redact position** (default ON) blanks Lat/Lon/Alt in the saved full log — share logs without publishing your rooftop.

Part 9 — Settings, the flash ring, and wear

Everything persistent lives in one **wear-levelled ring** in flash sector 7: 255 slots of 512 bytes. Each save (an **ES**, a successful **LC**, or a learned-data update) writes the next blank slot; the sector is erased only when the ring wraps — once per 255 saves. At the rate this firmware saves (even a busy bench hits ~73 saves/day) that is an erase every ~3.5 days, and the F411's flash survives ~10 000 erases per sector: **roughly 96 years**. You will never wear it out. **EW** shows the actual counts any time you're curious.

Two kinds of records share the ring:

- **Settings** (**ES** and friends) — everything you chose, saved only when you say so. Partial saves (**ES** **PID** etc.) write the *whole* settings block seeded from the last stored one, so saving one group can't clobber another.
- **Live data** — what the device learns by itself: the **LC** calibration, the learned drift/damping model, the last operating point. Saved automatically, but with hysteresis (only when something actually changed by a meaningful amount, and at most every 20 minutes).

Firmware upgrades: settings written by v1.04/v1.05 are migrated on recall (v4→v5 adds the algo-12 block at its defaults); a settings block from a radically different version, or a corrupt slot, is treated as "not there" — defaults, and the firmware asks for a calibration on the next boot. The ring itself self-heals: a slot half-written by a power cut fails its CRC and is simply skipped; the previous good slot wins.

The practical checklist, one last time:

- routine re-flash: safe, settings survive (Part 2),
- after a tuning session: **ES**,
- before selling/gifting a unit: **CR YES**,
- when a unit behaves strangely after an experiment session: **ER** (reload the saved, known-good settings), and only then start diagnosing.

Part 10 — Troubleshooting

Symptom	Likely cause → fix
Won't compile, ltoa error	STM32 core 3.0.0 — downgrade to 2.12.0 (Part 1.1)
Compiles, prints ? or garbage where numbers belong	Tools → C Runtime Library → Newlib Nano + Float printf/scanf
TFT white after core 3.0.0 build	same — core 2.12.0
TFT white with 2.x core	User_Setup.h wrong driver or TFT_MISO PA6 missing (Part 1.4)
Boot freezes right after TFT: init start	same as above — check wiring and the driver #define
USB serial missing after upload	press RESET; if still gone, Zadig driver for VID 0483/PID 5740 (Part 2.5)
Settings/calibration gone after flashing	someone used Erase Chip — that's the sector 7 rule (Part 2.2); re-run CT, LC, ES
Yellow LED off	no GPS fix — antenna, sky view, cable
HDOP:TIME never appears	survey-in not finished (needs timing receiver + SV 1 + good sky); it re-runs after each power loss
CT fails / rejects K	EFC wiring or oscillator sensitivity out of 0.02–2 mHz/LSB range — check the DAC buffer and EFC span
LC fails	run CT first; keep the unit still during the 5 minutes
Trend stuck NoCT (algo 12)	gain is auto but CT never ran → CT
Trend stuck ACQ / NoPL, algos 10–12	LPOL unset (the loop prints a warning and holds) — set LPOL -1 or 1, then ES LTIC
Algo 12 corrects constantly at level 0	noisy site — MF 1 (Alan's stored table), see Part 4.5
Phase ramps away after SAW 1	you don't have a timing receiver / qErr invalid — SAW 0
Report lines freeze when a program opens CDC but doesn't read	fixed in v1.06 (non-blocking writes + 1 KB queue); if you see it, you're on old firmware
Blue LED blinking rapidly	firmware fault caught (assert / stack overflow / heap) — read the serial message; report it
Unit resets spontaneously	check the 3V3 rail under OCXO load; the boot banner prints the reset cause
Display alive, no serial output	you're on Bluetooth only? 57600 on Serial2; or RP was typed — RR resumes

If none of that helps: capture a full log with the tuner (Part 8.5), note the **V** version string, and ask — with the log attached. A week of 1 Hz CSV is the difference between guessing and knowing.

Appendix A — How a GPSDO works, in plain words

You can use this device without reading this appendix — but the day something behaves oddly, this is the twenty paragraphs that will tell you *why*.

A.1 The problem

You want a 10 MHz signal that is right **now** (second to second) and right **forever** (over months and years). Two building blocks each deliver half of that, and neither delivers both:

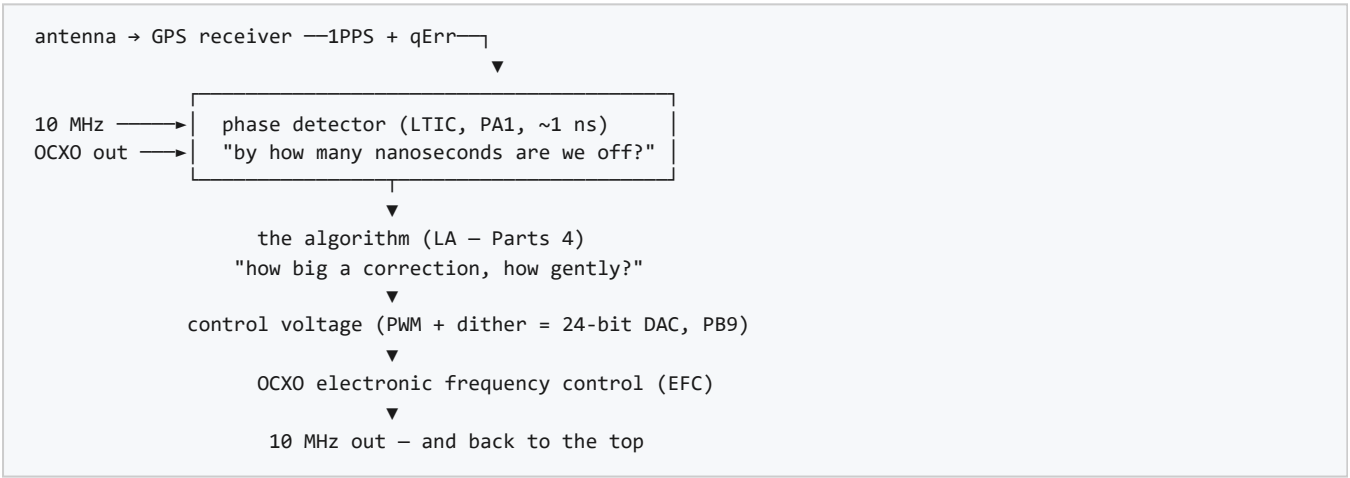
	Short term (seconds)	Long term (months)
OCXO alone	excellent — a good one is steady to parts in 10^{11}	drifts — aging and temperature pull it slowly off frequency
GPS 1PPS alone	noisy — each pulse lands within $\sim\pm 25$ ns of truth	excellent — it is steered by atomic clocks, forever

The oven part of "OCXO" is half the story already: the crystal sits in a little heated box held at one constant temperature (that's the 300-second warm-up at boot), because temperature is the crystal's biggest enemy. What remains is aging — a slow walk nobody can switch off.

The GPS pulse is the opposite: every individual second is only approximate (the receiver quantises time, the signal slows in the ionosphere, bounces off buildings), but the *average* over hours is nailed to atomic time, because the satellites carry atomic clocks and GPS cannot work otherwise.

A GPSDO is a division of labour: the OCXO handles the seconds, GPS handles the months, and a control loop in between moves the first, in tiny steps, to keep it anchored to the second.

A.2 The loop, piece by piece — and where each piece is on this board



- The phase detector answers "by how much are we late?" once a second, in nanoseconds. That single number — the *phase* — is the scoreboard the whole game is played on.
- The counter (TIM2) answers "are we fast or slow?" by counting the actual 10 MHz. It's coarse (0.01 Hz steps) but direct.
- The algorithm looks at those numbers and decides one thing: how much to nudge the control voltage, and over what time to spread the nudge.
- The DAC: 65536-step PWM, dithered to an effective 24 bits (steps of ~ 0.2 μ V — the oscillator notices far less than one PWM step, so the firmware remembers the fraction between steps).
- The EFC pin of the OCXO: volts in, frequency out — the whole adjustment range is only a few hertz wide, which is exactly why microvolts matter.

A.3 Phase, frequency, and the one multiplication you need

Frequency error and phase drift are the same fact in two costumes. If the oscillator is off by a fraction, its phase slips at exactly that rate:

Frequency error at 10 MHz	Phase slips
1 Hz	100 ns every second
0.01 Hz	1 ns every second
0.001 Hz (1 mHz)	1 ns every 10 seconds

Two consequences worth engraving:

1. If the phase stops moving, the frequencies are equal — whatever the phase value is. The loop's real goal is a *still* phase, parked near zero.
2. The errors worth chasing are absurdly small. One millihertz at 10 MHz is one part in 10^{10} — and the loop routinely resolves ten times better than that. This is why every wire, every microvolt and every degree matters more than in any other circuit you have built.

A.4 Why corrections must be gentle (the heart of the whole subject)

Suppose the GPS pulse arrives 20 ns late because the signal bounced off a building. The phase "error" you measure is 20 ns — but it isn't real: the oscillator didn't move. If the loop corrects immediately at face value, it **copies the GPS noise onto the oscillator** and the output ends up *worse* than the free-running OCXO. That single mistake is what separates a GPSDO from a GPS-flailing oscillator.

The defense is averaging: random noise shrinks by $\sqrt{N}$. Average 100 seconds $\rightarrow$ noise $\div 10$; average 1000 s $\rightarrow \div 31$. So the loop waits until the averaged GPS numbers are quieter than the oscillator's own drift, and only then acts — with a correction sized to what the long average actually proved, spread over a comparable time. That is why:

- algorithm 11 has `LTC` (a time constant — "how patient is this loop"),
- algorithm 12 lets **the size of the error choose its own averaging time** (a big error is proven in seconds and acted on in seconds; a tiny one is proven over an hour and corrected over an hour),
- every correction in this firmware is dithered into sub-LSB steps rather than dumped at once.

One free lunch: **sawtooth correction (SAW 1)**. The receiver *knows* how far it quantised each pulse (the `qErr` number) and confesses it every second. Subtracting the confession turns ± 25 ns of fake error into a few ns before any averaging even starts.

A.5 Holdover

When GPS disappears (antenna cut, receiver dead), the loop has nothing true to steer by. The right move is to **freeze** the last good control voltage and let the OCXO coast on its own stability — that's holdover (`MH` manual, or automatic on fix loss; the yellow LED blinks, the display goes orange, `[HOLDOVER]` replaces the trend in the report). The oscillator then drifts with aging and temperature — slowly, but unstoppably, until GPS returns.

A.6 How results are judged: Allan deviation in one paragraph

People compare frequency standards with **Allan deviation (ADEV)**: roughly "how much does the average disagree with itself, taken over τ seconds", plotted against τ . A GPSDO plot falls from left to right: at $\tau = 1$ s you see the oscillator's own noise; as τ grows, averaging beats the noise down and the GPS anchoring takes over (on the author's bench: $\sim 3 \cdot 10^{-9}$ at 1 s falling to $\sim 5 \cdot 10^{-12}$ at 3000 s). When someone posts "1e-12 at tau 10 000", that is the sentence they are saying. A *hump* in the middle of the curve means the loop is fighting itself — too fast for its own noise — and is the classic signature of a badly tuned GPSDO.

Appendix B — PID for the reluctant

You never *need* this appendix to use the device — `CT` tunes the loops for you. This is for the day you open the **PID algo** 3-9 tab in the tuner, or type `KP`, and want to know which lever you are holding and which way it bites. No mathematics beyond multiplication is used.

B.1 The shower picture

Every control loop ever built does the same four things:

- 1. **Measure** where you are (water temperature).
- 2. **Compare** with where you want to be (comfortable).
- 3. The difference is the **error** (too cold by 5 degrees).
- 4. **Act** (open the hot tap), then wait and repeat.

The only question in the whole subject is step 4: *how much do you open the tap?* PID — three letters, three answers — is the standard recipe.

B.2 P — proportional: "react to now"

Open the tap in proportion to the error. Cold by a lot → open a lot. Cold by a little → open a little.

- **P alone has a flaw:** it needs some error to produce any action at all, so it settles *near* the target, never at it — the last degree of cold isn't enough to hold the tap open. That residual is called **droop**.
- **Too much P:** you overshoot hot, then overcorrect cold, and oscillate between the two — the shower-from-hell.
- **Too little P:** takes forever to get anywhere.

B.3 I — integral: "remember the past"

Watch the error *over time*. Even a tiny persistent coldness accumulates, in a running sum, until the loop adds enough action to close it. **I is what kills the droop** — it is the "eventually, exactly right" term.

- **Too much I:** the classic slow yo-yo. The sum builds up during the approach, then spends itself as overshoot, rebuilds the other way, and the loop swings gently for ages.
- **Windup:** if the tap is already fully open (the output is at a limit) the sum keeps growing uselessly and then takes forever to unwind. The fix is a clamp on the sum — that is exactly what `IL` (`I_LIMIT`) is.

B.4 D — derivative: "anticipate"

React not to the error but to how *fast* it is changing. Warming quickly? Start closing the tap *before* you arrive. **D is the damper** — the suspension that stops the oscillations P and I would otherwise love.

- **D's vice:** it amplifies measurement noise (differentiating a jittery sensor produces spikes). That's why D terms are usually filtered, small, or both — and why a noisy phase measurement makes D-heavy tuning worse, not better.
- In the PLL algorithms (4/5/7) the `Kd` slot acts on the accumulated *phase* rather than a raw derivative — same damping duty, different wiring. Don't let the letter confuse you; think "the damping knob".

B.5 The three letters, one table

Knob	Reacts to	Cures	Too much →	Too little →
P	error right now	sluggishness	overshoot, fast oscillation	takes forever
I	error accumulated over time	standing offset (droop)	slow yo-yo oscillation, windup	settles near, never at, the target
D	how fast error changes	overshoot (damping)	nervous jitter chasing noise	overshoot rings

B.6 Where the knobs live in this firmware

You are turning	Where	Which letter it really is
KP n va1 / KI / KD / IL n va1	algorithms 3–9 (tuner tab PID algo 3-9)	literally P, I, D and the integrator clamp
AQP...AQL, DPP...DPL, LKP...LKL	algorithm 10's three stages (tab LTIC)	a full PID set per stage — the loop re-tunes itself as it settles
LG / LD / LTC	algorithm 11 (tab LTIC-Lars)	gain (how hard), damping (how it settles), time constant (how patient) — the same three knobs wearing Lars' clothes
MG and the limits	algorithm 12 (tab Multi-level)	no PID at all — see below

Algorithm 12 deserves one honest paragraph: it has **no P, I or D**. Instead of fixed gains it asks, each time, "how big is the error, how long did I average to prove it?" and scales the correction to match — big errors get fast, firm treatment; small ones get slow, gentle treatment. It is the same physics with the tuning built in, which is why its only hand-knobs are the gain (MG, and CT measures that for you) and the limits that decide what counts as "big".

B.7 Why CT exists — the ruler problem

PID numbers live in units of "PWM steps per hertz of error". But one PWM step is worth a *different number of hertz on every individual oscillator* — 0.32 mHz on one of the author's Vectrons, nearly seven times less on a narrow-EFC build. Without measuring your oscillator, the same `Kp = 1000` is a gentle nudge on one board and a violent shove on another. **CT measures your oscillator's volts-to-hertz response and rescales every gain to match**, so the shipped tuning means the same physical thing on every unit. This is also why the manual's calibration order is law: CT first, LC second, tuning (if ever) last.

B.8 Ten rules of thumb

1. Run CT and LC before touching any gain. In most lives, that's the end of tuning.
2. Change **one** knob at a time.
3. Halve or double — never $\times 10$. Loop tuning reacts to ratios, not arithmetic.
4. Give every change an hour. These loops average over minutes; judging a change after thirty seconds is reading a book by one letter.
5. When in doubt, go **slower** (bigger LTC, smaller gains). Slow is merely unexciting; fast is unstable.
6. Oscillation → cut P first, then I.
7. An offset that never quite closes → more I — or, more likely, you skipped CT.
8. Output visibly chasing GPS noise → slower loop, SAW 1, better antenna view. Not bigger damping.
9. ES after every session; ER un-experiments a bad afternoon.
10. If it fights you for a whole day, suspect hardware before tuning: antenna sky view, temperature (heaters, doors, sunlight), EFC wiring. The loops in this firmware are hard to break and easy to blame.